TP2 : Structures de données et écriture idiomatique

Enseignant : Abderrahmane Kasmi — **Public :** 1ère année Cycle Ingénieur ISEN

1 Introduction

Ce TP a pour objectif de vous familiariser avec les principales structures de données en Python et de vous apprendre à les utiliser de manière efficace. Il vise également à vous initier à l'écriture pythonique, en adoptant les bonnes pratiques et les idiomes propres au langage Python.

Objectifs du TP

À l'issue de ce TP, vous serez capable de :

- Manipuler les collections natives (listes, tuples, ensembles, dictionnaires) et comprendre leurs propriétés d'immutabilité et de hashabilité.
- Écrire des compréhensions complexes de listes, ensembles et dictionnaires.
- Exploiter les fonctions de haut niveau (`map`, `filter`, `sorted`) avec des expressions `lambda`.
- Construire des flux de données efficaces au moyen de générateurs (`yield`) et manipuler le mécanisme des itérateurs.
- Refactoriser du code impératif verbeux en du code pythonique, performant et lisible.

2 Collections

2.1 Exercice 1 — Analyse de catalogue produit et droits d'accès

1. Un serveur reçoit des identifiants d'utilisateurs sous forme de liste avec des doublons dus à des connexions multiples :

```
logs_connexions = ["user_102", "user_405", "user_102", "user_88", "
    user_405", "user_301"]
```

   Isolez la liste des utilisateurs uniques.

2. On gère les compétences techniques requises pour deux projets d'ingénierie :

```
projet_alpha = {"Python", "Linux", "Docker", "SQL"}
projet_beta = {"Python", "AWS", "Kubernetes", "Docker"}
```

   Affichez :
   - Les compétences communes aux deux projets.
   - L'ensemble de toutes les compétences requises sans doublons.
   - Les compétences spécifiques à Alpha qui ne sont pas nécessaires pour Beta.

3. Représentez un panier d'achat sous forme de dictionnaire : `panier = {"SSD 1TB": 89.99, "RAM 16GB": 65.0, "Clavier": 45.50}`.

— Ajoutez un article `"Souris"` à 25.0 .
— Augmentez le prix du `"SSD 1TB"` de 10 %.
— Récupérez le prix d'un article `"Ecran"` en évitant une erreur `KeyError` si l'article n'existe pas (fournir une valeur par défaut de `0.0`).

3 Compréhensions

3.1 Exercice 2 — Traitement de données météorologiques

1. Soit le relevé de températures en degrés Celsius au cours d'une semaine :

```
temperatures = [18.5, 22.0, 14.2, 26.5, 19.8, 11.0, 25.0]
```

— Générez une nouvelle liste contenant uniquement les jours chauds ($> 20.0\,°C$).
— Créez une liste d'étiquettes : `"Chaud"` si $T \geq 20.0$, sinon `"Frais"`.

2. Soient les données suivantes relatives aux mesures d'un capteur :

```
capteurs = ["Sensor_A", "Sensor_B", "Sensor_C", "Sensor_D"]
valeurs = [102.4, 88.1, 120.5, 95.0]
```

— Créez un dictionnaire associant chaque capteur à sa valeur uniquement si la valeur est supérieure à 90.0, en utilisant une compréhension de dictionnaire et `zip`.
— Inversez ce dictionnaire (valeurs comme clés, capteurs comme valeurs).

4 Fonctions de haut niveau

4.1 Exercice 3 — Filtrage et tri de données financières

1. Soit une liste de transactions en dollars USD : `transactions = [120.0, -45.5, 300.0, -12.0, 85.0]`.

— Utilisez `filter()` et une fonction `lambda` pour ne conserver que les crédits (valeurs positives).
— Utilisez `map()` et une lambda pour convertir l'ensemble des montants en Euros (taux : $1USD = 0.92EUR$).

2. On dispose d'une liste de collaborateurs enregistrés sous forme de dictionnaire :

```
employes = [
    {"nom": "Sophie", "anciennete": 5, "salaire": 42000},
    {"nom": "Alexandre", "anciennete": 2, "salaire": 35000},
    {"nom": "Elena", "anciennete": 8, "salaire": 58000},
    {"nom": "Marc", "anciennete": 5, "salaire": 39000}
]
```

— Triez cette liste par ordre décroissant de salaire à l'aide de `sorted()` et d'une clé `lambda`.
— Triez la liste selon un critère multiple : d'abord par ancienneté croissante, puis par salaire décroissant en cas d'égalité d'ancienneté.

5 Itérateurs et générateurs

5.1 Exercice 4 — Découverte des itérateurs

1. Soit le tuple suivant représentant les étapes d'un pipeline de données :

```
etapes = ("Ingestion", "Nettoyage", "Transformation", "Export")
```

   — Créez un objet itérateur explicite à partir de ce tuple en utilisant la fonction intégrée `iter()`.
   — Consommez manuellement cinq éléments à l'aide de la fonction `next()`.

5.2 Exercice 5 — Générateur de la suite de Fibonacci sous contrainte de longueur

1. Écrivez une fonction générateur `fibonacci_sequence(longueur)` qui utilise l'instruction `yield` pour produire les termes de la suite de Fibonacci ($F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$) jusqu'à générer exactement le nombre de termes spécifié par l'argument `longueur`.
2. Appelez ce générateur dans une boucle pour afficher les 12 premiers termes de la suite ($N = 12$).
3. Vérifiez le comportement de votre générateur si la longueur demandée est égale à 0 ou 1.

Principe de mémoire du générateur

Un générateur suspend son exécution à chaque rencontre du mot-clé `yield` en sauvegardant l'état local de ses variables, puis la reprend là où elle s'était arrêtée lors de la demande du terme suivant.

6 Écriture idiomatique

6.1 Exercice 6 — Refactorisation d'un gestionnaire de notes

Le script suivant gère un registre d'étudiants sous forme de dictionnaire de listes. Il vérifie la présence de clés, calcule les moyennes, identifie le major et prépare une liste d'admis de manière impérative et verbeuse.

```
etudiants = [
    ("Dupont", "Alice", [14.0, 16.5, 12.0]),
    ("Martin", "Bob", [8.0, 9.5, 10.0]),
    ("Bernard", "Charlie", [15.0, 18.0, 16.0])
]

# 1. Verification de pr sence du dictionnaire de profils
registre = {}
for i in range(len(etudiants)):
    nom = etudiants[i][0]
    prenom = etudiants[i][1]
    notes = etudiants[i][2]

    # Calcul manuel de la moyenne
    total = 0
    for n in notes:
        total = total + n
```

```
    moyenne = total / len(notes)

    registre[nom] = {
        "prenom": prenom,
        "moyenne": moyenne
    }

# 2. Recherche du statut de pr sence du champ optionnel 'option'
for nom in registre:
    if "option" in registre[nom]:
        opt = registre[nom]["option"]
    else:
        opt = "Aucune"
    registre[nom]["option"] = opt

# 3. Construction de la liste des admis (moyenne >= 10)
admis = []
for nom in registre:
    if registre[nom]["moyenne"] >= 10.0:
        admis.append(nom)

# 4. Verification de contenu de la liste
if len(admis) > 0:
    print("Nombre d'admis :", len(admis))
```

6.1.1 Consignes de refactorisation

Réécrivez ce script en adoptant un style idiomatique Python :

1. Parcourez la liste `etudiants` à l'aide du dépaquetage de tuples (*unpacking*) direct dans la boucle `for`, sans utiliser d'indexation explicite `range(len(...))`.
2. Calculez la moyenne à l'aide des fonctions intégrées `sum()` et `len()`.
3. Utilisez la méthode `dict.get()` pour fournir la valeur par défaut `"Aucune"` de l'option sans employer de condition `if/else`.
4. Construisez le dictionnaire des admis via une compréhension de dictionnaire.
5. Remplacez la condition `if len(admis) > 0` par le test direct de la valeur de vérité (*truthiness*) de la collection.